14-1 在Linux上安装Python

【环境】

- 腾讯云服务器 Centos 8

【安装方式】

- 源码编译安装

安装步骤：

第1步：更新yum源

```
cd /etc/yum.repos.d/
sed -i 's/mirrorlist/#mirrorlist/g' /etc/yum.repos.d/CentOS-*
sed -i 's|#baseurl=http://mirror.centos.org|baseurl=http://vault.centos.org|g'
/etc/yum.repos.d/CentOS-*
wget -O /etc/yum.repos.d/CentOS-Base.repo
https://mirrors.aliyun.com/repo/Centos-vault-8.5.2111.repo
yum clean all
yum update

# 安装编译依赖
yum install -y libffi-devel zlib-devel bzip2-devel openssl-devel ncurses-devel
sqlite-devel readline-devel tk-devel gdbm-devel xz-devel
```

- 第2步：从官网下载Python源码包（如果下载比较慢，也可以在本地下载好传到远程服务器上）

```
[root@VM-4-2-centos ~]# pwd
/root
[root@VM-4-2-centos ~]# wget https://www.python.org/ftp/python/3.10.4/Python-
3.10.4.tgz
```

- 第3步：解压缩

```
[root@VM-4-2-centos ~]# tar -xzvf Python-3.10.4.tgz
[root@VM-4-2-centos ~]# ls
Python-3.10.4  Python-3.10.4.tgz
```

- 第4步：编译安装，--prefix指定安装目录

```
cd Python-3.10.4
./configure --prefix=/usr/local/bin/python310

make && make install
```

- 第5步：创建软连接，创建软连接后才能在任何路径下使用命令(可选)

```
ln -s /usr/local/bin/python310/bin/python3.10 /usr/bin/python3.10
ln -s /usr/local/bin/python310/bin/pip3.10 /usr/bin/pip3.10
```

14-2 在Linux上部署Web项目

- 开发完成的web项目可以在本地部署，部署方式：`uvicorn main:app --host 127.0.0.1 --port 8080`
- 其中：--host指定部署在那个IP地址，--port表示服务监听的端口号。
- 但是，本地的部署只能在本地自己访问，别人无法访问；想要让别人可以访问，需要将服务部署在公网上。
- 比如，将web项目部署在阿里云服务器，腾讯云服务器上。

部署准备：

- 准备云服务器一台，比如centos8（购买云服务器参考：https://www.51zxw.net/List.aspx?cid=972）
- 如果不想使用云服务器，只想体验在linux上部署服务，可以使用虚拟机。
- 远程连接工具，xshell、xftp等等。（课程素材中有提供）
- web项目（简单演示）

```python
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def index():
    return {"message": "Hello World"}
```

部署流程

- 第1步：使用xshell连接远程服务器，进入/root（如果是root用户，进去后默认就是/root目录）
- 第2步：在linux服务器上安装python3.10，安装项目的依赖，比如： fastapi、uvicorn等
- 第3步：确保项目在本地可以跑起来，`uvicorn main:app --host 127.0.0.1 --port 8080`
- 第4步：使用xftp将本地的main.py文件拷贝到远程服务器的 /root目录下
- 第5步：使用命令启动服务，注意 --host这里不要使用 0.0.0.0，也不要使用公网IP

```
uvicorn main:app --host 0.0.0.0 --port 8080     # --host 0.0.0.0

# 如果上述命令报错，则需要添加环境变量
export PATH="/usr/local/bin/python310/bin:$PATH"
```

- 第6步：在云服务器的管理页面的安全组或者防火墙，放行8080端口
- 第7步：在本地浏览器，输入云服务器的公网IP:8080，然后回车， 即可访问服务

```
http://公网IP:8080
```

14-3 使用systemd部署

直接使用上述部署方式，是一种前台运行方式，一旦退出shell，则服务就挂掉了，因此我们需要使用一种后台运行的方式。

本节课，给大家接收一种常用的后台运行方式，即使用systemd部署服务。

部署流程

- 第一步：创建服务

```
cd /etc/systemd/system/

vim simple_app.service          # 新建名为 simple_app.service的文件，将并将下面的内容拷贝进去然后保存退出


[Unit]
Description=Simple app Service


[Service]
Type=oneshot
ExecStart=/usr/local/bin/python310/bin/python3.10 /root/main.py
StandardOutput=syslog
StandardError=inherit
```

```
[Install]
WantedBy=multi-user.target
```

- 第二步：调整代码

```
# /root/main.py
import uvicorn
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def index():
    return {"message": "Hello World"}

uvicorn.run(app, host="0.0.0.0", port=8080)
```

- 开启服务

```
systemctl enable simple_app.service
```

- 启动服务

```
systemctl start simple_app.service
```

- 关闭服务

```
systemctl stop simple_app.service
```

- 查看运行状态

```
systemctl status simple_app.service
```

14-4 使用docker部署

使用systemd部署其实已经非常好的管理服务了，但是你会发现它依赖本地环境，比如依赖本地Pyhton解释器等等。

如果想要一个完全独立的运行环境，我们可以使用容器化的方式部署我们的服务。

这节课，我们就来看下如何使用Docker的方式部署服务。

还是针对这个简单的服务

```python
# /root/main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def index():
    return {"message": "Hello World"}
```

部署前提，安装好docker，参考安装视频：https://www.51zxw.net/Show.aspx?cid=972&id=111447

使用Docker部署流程

- 第1步，调整项目结构

```
- docker_app
    |- main.py              # fastapi的服务
    |- requirements.txt      # 项目依赖，比如 fastapi/uvicorn等
```

- 第2步：生成 requirements.txt

```
pip3.10 freeze > ./requirements.txt
```

- 第3步：编写Dockerfile

```dockerfile
FROM python:3.10-slim

COPY . /var/www/src
WORKDIR /var/www/src

RUN pip install -r requirements.txt -i
https://pypi.tuna.tsinghua.edu.cn/simple

CMD ["/usr/local/bin/uvicorn", "main:app", "--host", "0.0.0.0", "--port",
"8080"]
```

- 第4步：构建镜像

```
docker build -t docker_app:v1.0 .
```

- 第5步：运行容器

```
docker images

docker run -d --rm -p8080:8080 镜像id
```

【提示：使用容器部署时记得先把上节课的 systemd部署的服务关了，避免都是8080端口冲突了】

14-5 使用docker-compose单机部署

- 使用docker可以方便的部署我们的服务，并且可以做到相对环境隔离。
- 不过你可能会遇到以下场景：
  - 一个web服务需要依赖mysql数据库
  - 此时如果使用docker的部署方式，那我们即需要一个一个的部署三个容器
  - 并且，因为是容器化部署，所以每次新建容器的Ip地址都是变化的，那web容器如何连接mysql容器？
  - 此时，我们就需要一个容器编排工具，而docker-compose就是一个官方推荐的单机编排工具。

前提：在服务器上安装好docker-compose，参考视频：https://www.51zxw.net/Show.aspx?cid=972&id=111532

准备项目

- 为了显示出docker-compose的容器编排功能，我们以 第十章整理的blog_app项目为例。

docker-compose部署项目流程

- 第一步：准备好requirements.txt 【参考上节课】
- 第二步：编写 Dockerfile 【参考上节课】
- 第三步：编写docker-compose.yml

```
version: "3.9"
services:
  mysql:
    image: percona:5.7
    environment:
      MYSQL_DATABASE: app
      MYSQL_USER: app
      MYSQL_PASSWORD: app
      MYSQL_ROOT_PASSWORD: app
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
  web:
    build: .
    volumes:
      - .:/var/www/src
    depends_on:
      - mysql
    command: sh -c "/usr/local/bin/uvicorn main:app --reload --host=0.0.0.0 --port=8080"
    ports:
```

```
      - "0.0.0.0:8080:8080"
```

- 第四步：调整数据库连接

```python
# darabase.py


import pymysql

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session

pymysql.install_as_MySQLdb()


# 根据docker-compose.yml中mysql的环境变量做如下修改：
# 用户名改为 app
# 密码改为 app
# 数据库IP改为 mysql
# 数据库名改为 app
DATABASE_URL = "mysql://app:app@mysql:3306/app"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)


def get_db():
    db: Session = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- 第五步：启动服务（内部会自动构建镜像和运行服务）

```
docker-compose up -d     # 启动服务
docker-compose ps         # 查看容器运行状态
docker-compose restart    # 重启服务
docker-compose stop      # 暂停服务
docker-compose down      # 彻底删除服务
```

【完整代码见课程素材】

- 注意:因为没有在数据库中创建表，所以接口调用时会报错。